Instituto Hardware BR
Programa EmbarcaTech

Projeto 1:
Controle de Acesso (Check-In):
DOCUMENTAÇÃO

Adriana Rocha Castro de Paula
Luana Maria da Silva Menezes

Turma Campinas - 2026

Sumário

1. SISTEMA EMBARCADO **5**
1.1. Introdução 5
1.1.1. Contextualização 5
1.1.2. Definição do Problema 5
1.1.3. Objetivos 5
1.1.4. Justificativa 6
1.2. Fundamentação Teórica 6
1.2.1. Sistemas Embarcados 7
1.2.2. Sistemas Operacionais de Tempo Real (RTOS) 7
1.2.3. Arquitetura de Software em Camadas 7
1.2.4. Padrões de Projeto Utilizados 8
1.2.5. Interface Gráfica Embarcada 8
1.3. Metodologia 9
1.3.1. Abordagem de Desenvolvimento 9
1.3.2. Ferramentas Utilizadas 9
1.3.3. Bibliotecas Externas 9
1.4. Especificação de Requisitos 10
1.4.1. Requisitos Funcionais 10
1.4.2. Requisitos Não-Funcionais 11
1.5. Arquitetura do Sistema 11
1.5.1. Arquitetura de Hardware 11
1.5.2. Arquitetura de Software 12
1.6. Implementação 13
1.6.1. Estrutura do Projeto 13
1.6.2. Controle de Versão 15
1.7. Testes e Validação 15
1.7.1. Estratégia de Testes 15
1.7.2. Demos Standalone 16
1.7.3. Testes Integrados 16
1.8. Resultados 16
1.8.1. Funcionalidades Implementadas 16
1.8.2. Nível de Maturidade Tecnológica (TRL) 17
2. AGENDAMENTOS DE CONSULTAS POR WEB API E BOT DO TELEGRAM **18**
2.1. INTRODUÇÃO 18
2.2. ARQUITETURA DO SISTEMA 19
2.2.1. Componentes Principais 19
2.2.2. Stack Tecnológico 19
2.3. API REST 19
2.3.1. Visão Geral 19
2.3.2. Autenticação e Segurança 19

2.3.3. Estrutura de Dados 20
2.3.3.1. Pacientes 20
2.3.3.2. Categorias 20
2.3.3.3. Profissionais 20
2.3.3.4. Horários Profissionais 20
2.3.3.5. Consultas 21
2.4. BOT TELEGRAM 21
2.4.1. Visão Geral 21
2.4.2. Funcionalidades 21
2.4.2.1. Comandos Disponíveis 21
2.4.3. Fluxo de Agendamento 22
3. INTEGRAÇÃO WEB API COM SISTEMA EMBARCADO 23
3.1. Visão Geral da Integração 23
3.2. Arquitetura de Integração 23
4. PROTÓTIPO FÍSICO 23
5. TRABALHOS FUTUROS 24
6. CONCLUSÃO 25
7. APÊNDICE A: Manual do Usuário 26
7.1. Manual do usuário do bot de agendamento 26
7.1.1. Sobre o Bot 26
7.1.2. Como Iniciar 26
7.1.3. Como Agendar uma Consulta 26
7.1.4. Como Recuperar o QR Code de uma Consulta 28
7.1.5. Capturas de Tela do Agendamento por Telegram 28
7.2. Manual do usuário Totem de Check-in (Sistema Embarcado) 30
8. APÊNDICE B: Documentação Técnica 33
8.1. Protocolos de Comunicação 33
8.2. Detalhes de Implementação (HAL, Drivers, Serviços, Tasks) 34
8.2.1. Camada HAL (Hardware Abstraction Layer) 34
8.3. Camada de Drivers 35
8.4. Camada de Serviços 36
8.5. Camada de Aplicação 38
8.6. API REST Completa (endpoints, banco) 40
8.6.1. Endpoints da API 40
8.6.1.1. Autenticação 40
8.6.1.2. Pacientes 40
8.6.1.3. Categorias 41
8.6.1.4. Profissionais 42
8.6.1.5. Horários Profissionais 43
8.6.1.6. Consultas 43
8.6.1.7. QR Code 45
8.6.2. Tratamento de Erros 45

8.6.3. Padrões Arquiteturais 46
8.6.4. BANCO DE DADOS 46
8.6.4.1. Sistema de Gerenciamento 46
8.6.4.2. Diagrama de Relacionamentos 46
8.6.4.3. Migrations 46
8.7. BOT TELEGRAM 46
8.7.1. Integração com API 46
8.7.2. Tratamento de Erros 47
8.7.3. Experiência do Usuário 47
8.8. Integração (detalhes de comunicação embarcado-API) 47
8.8.1. Endpoints Relevantes para Integração 47
8.8.2. Protocolo de Comunicação 48
8.8.3. Segurança na Integração 48
8.9. Protótipo Físico - Fotos 49
8.10. Capturas de Tela do Sistema 51
9. REFERÊNCIAS 56

1. SISTEMA EMBARCADO

1.1. Introdução

1.1.1. Contextualização

O setor de saúde enfrenta desafios constantes relacionados à gestão do fluxo de pacientes em clínicas e consultórios médicos. O processo tradicional de check-in, realizado manualmente por recepcionistas, frequentemente resulta em filas de espera, erros de identificação e sobrecarga da equipe administrativa. Com o avanço da tecnologia embarcada e a popularização de dispositivos de autoatendimento, torna-se viável o desenvolvimento de soluções que automatizam e agilizam esse processo.

Neste contexto, os sistemas embarcados apresentam-se como uma alternativa de baixo custo e alta confiabilidade para a implementação de totens de autoatendimento. A utilização de microcontroladores modernos, como o Raspberry Pi Pico W, aliada a sistemas operacionais de tempo real (RTOS), permite o desenvolvimento de aplicações responsivas e robustas, capazes de operar de forma autônoma em ambientes clínicos.

1.1.2. Definição do Problema

O processo convencional de check-in em clínicas médicas apresenta diversas limitações:

- Filas de espera: Pacientes aguardam atendimento na recepção, especialmente em horários de pico;
- Erros de identificação: A conferência manual de documentos está sujeita a falhas humanas;
- Sobrecarga da recepção: Funcionários dividem atenção entre check-in, atendimento telefônico e outras demandas;
- Experiência do paciente: O processo burocrático impacta negativamente a percepção de qualidade do atendimento.

1.1.3. Objetivos

Objetivo Geral

Desenvolver um sistema embarcado de autoatendimento para check-in de pacientes em clínicas médicas, utilizando microcontrolador Raspberry Pi Pico W, capaz de validar agendamentos através da leitura de QR Code, autenticar pacientes por biometria digital e exibir informações da consulta em interface gráfica touchscreen.

Objetivos Específicos

- Projetar e implementar arquitetura de software em camadas que promova modularidade e manutenibilidade
- Desenvolver drivers de hardware para integração de display TFT, touchscreen capacitivo, leitor de QR Code e sensor biométrico
- Implementar comunicação com API REST para download de agendamentos e sincronização de dados
- Utilizar sistema operacional de tempo real (FreeRTOS) para gerenciamento de tarefas concorrentes
- Desenvolver interface gráfica intuitiva utilizando a biblioteca LVGL
- Implementar cadastro e verificação de biometria digital com sincronização ao servidor
- Implementar persistência local de dados em cartão SD para operação com conectividade reduzida
- Validar o sistema através de testes integrados com hardware real

1.1.4. Justificativa

A escolha do Raspberry Pi Pico W como plataforma de desenvolvimento justifica-se por seu baixo custo, disponibilidade no mercado brasileiro, capacidade de processamento adequada (dual-core ARM Cortex-M0+ a 133 MHz), conectividade WiFi integrada e amplo suporte da comunidade. A utilização de FreeRTOS permite o desenvolvimento de uma aplicação multitarefa responsiva, essencial para um sistema que deve reagir em tempo real às interações do usuário.

Do ponto de vista acadêmico, o projeto integra conhecimentos de diversas áreas da engenharia de sistemas embarcados: programação de baixo nível, protocolos de comunicação (SPI, I2C, UART), sistemas operacionais de tempo real, arquitetura de software, interfaces homem-máquina e integração de sistemas. Esta abordagem multidisciplinar proporciona uma experiência de aprendizado abrangente e alinhada com as demandas do mercado de trabalho.

Do ponto de vista prático, o sistema proposto pode ser replicado a baixo custo por clínicas de pequeno e médio porte, contribuindo para a modernização do atendimento em saúde e democratização do acesso a tecnologias de autoatendimento.

1.2. Fundamentação Teórica

Esta seção apresenta os conceitos teóricos fundamentais que embasam o desenvolvimento do sistema de check-in, abrangendo sistemas embarcados, sistemas operacionais de tempo real, arquitetura de software e padrões de projeto.

1.2.1. Sistemas Embarcados

Um sistema embarcado é um sistema computacional dedicado a executar funções específicas, geralmente integrado a um dispositivo maior. Diferentemente de computadores de propósito geral, sistemas embarcados são projetados para realizar tarefas bem definidas, com restrições de recursos como memória, processamento e consumo de energia.

O Raspberry Pi Pico W utilizado neste projeto é baseado no microcontrolador RP2040, que possui processador dual-core ARM Cortex-M0+ operando a 133 MHz, 264 KB de memória SRAM e 2 MB de memória Flash externa. O dispositivo conta ainda com conectividade Wi-Fi 802.11n integrada por meio do chip CYW43439, além de diversas interfaces de comunicação, incluindo duas UART, duas SPI e duas I2C, totalizando 26 pinos GPIO multifuncionais. Também oferece suporte a DMA, permitindo transferências de dados mais eficientes.

1.2.2. Sistemas Operacionais de Tempo Real (RTOS)

Um Sistema Operacional de Tempo Real (RTOS) é projetado para processar eventos dentro de prazos determinísticos, permitindo o gerenciamento eficiente de múltiplas tarefas concorrentes em sistemas embarcados. O FreeRTOS, adotado neste projeto, oferece recursos como gerenciamento de tarefas com prioridades, comunicação por filas, sincronização por semáforos e mutexes, grupos de eventos e timers por software. Sua escolha se deve à ampla adoção na indústria, suporte ao RP2040 pela Raspberry Pi Foundation, baixo consumo de memória e extensa documentação disponível.

1.2.3. Arquitetura de Software em Camadas

A arquitetura em camadas (Layered Architecture) é um padrão de projeto que organiza o software em níveis hierárquicos, onde cada camada possui responsabilidades bem definidas e comunica-se apenas com as camadas adjacentes. Este padrão promove:

- Separação de responsabilidades: Cada camada encapsula uma funcionalidade específica;
- Baixo acoplamento: Alterações em uma camada têm impacto mínimo nas demais;
- Testabilidade: Camadas podem ser testadas isoladamente através de mocks;
- Portabilidade: A troca de hardware requer alterações apenas nas camadas inferiores;
- Manutenibilidade: O código torna-se mais organizado e fácil de compreender.

No contexto de sistemas embarcados, uma arquitetura típica em camadas inclui:

- Camada de Hardware (HAL): Abstrai o acesso direto aos periféricos do microcontrolador;
- Camada de Drivers: Implementa a lógica de comunicação com dispositivos externos;

- Camada de Serviços: Contém a lógica de negócio da aplicação;
- Camada de Aplicação: Gerencia o fluxo da aplicação, interface com usuário e orquestração.

1.2.4. Padrões de Projeto Utilizados

O desenvolvimento do sistema emprega diversos padrões de projeto consagrados na engenharia de software:

Singleton: Garante que uma classe possua apenas uma instância em toda a aplicação, fornecendo um ponto de acesso global a ela. Este padrão é utilizado nos serviços do sistema (AppointmentService, DataStorageService, TimeService, NetworkSyncService, FingerprintService) e na máquina de estados, garantindo consistência de estado e evitando conflitos de acesso.

State Machine (Máquina de Estados): Modela o comportamento do sistema como um conjunto finito de estados e transições. A máquina de estados gerencia 11 estados distintos (INITIALIZING, DOWNLOADING_APPOINTMENTS, IDLE, VALIDATING, FINGERPRINT_VERIFYING, FINGERPRINT_ENROLLING, FINGERPRINT_UPLOADING, APPOINTMENT, ERROR, ERROR_CRITICAL, RESTARTING). Este padrão é fundamental para sistemas embarcados reativos, pois facilita a visualização do comportamento, torna explícitas todas as condições possíveis, simplifica a adição de novos estados e previne estados inválidos.

Adapter: Permite que interfaces incompatíveis trabalhem juntas, convertendo a interface de uma classe para outra esperada pelo cliente. No projeto, este padrão é utilizado para integrar a biblioteca LVGL com os drivers de display e touch através das classes LVGLDisplayAdapter e LVGLTouchAdapter, desacoplando a biblioteca gráfica dos drivers específicos de hardware.

Dependency Injection: Implementado de forma simplificada através de injeção por construtor, os drivers recebem suas dependências de HAL através do construtor, permitindo a substituição de implementações concretas para testes ou portabilidade para outras plataformas.

1.2.5. Interface Gráfica Embarcada

A **LVGL** (Light and Versatile Graphics Library) é uma biblioteca gráfica open-source voltada a sistemas embarcados com recursos limitados, oferecendo renderização eficiente com anti-aliasing, diversos widgets, sistema de estilos e animações, baixo consumo configurável de memória e independência de hardware por meio de drivers abstratos.

1.3. Metodologia

1.3.1. Abordagem de Desenvolvimento

O desenvolvimento seguiu uma abordagem incremental e iterativa, com ciclos de implementação, testes em hardware e refinamento. Cada módulo foi testado individualmente antes da integração ao sistema principal. A metodologia adotada consistiu nas seguintes etapas:

1. **Prototipação de hardware:** Montagem e teste individual de cada periférico em protoboard;
2. **Desenvolvimento de drivers:** Implementação bottom-up (HAL → Driver → Adapter);
3. **Montagem do protótipo físico:** Construção da versão soldada do sistema com conectores para facilitar manutenção;
4. **Implementação de serviços:** Lógica de negócio independente de hardware;
5. **Integração progressiva:** Adição gradual de funcionalidades ao sistema principal via tasks FreeRTOS;
6. **Testes integrados:** Validação do fluxo completo em hardware real.

Padrões de linguagem adotados: C11 para código de baixo nível (HAL, drivers) e C++17 para camadas superiores (serviços, aplicação).

1.3.2. Ferramentas Utilizadas

VS Code: IDE com extensões para Raspberry Pi Pico, C/C++ e CMake Tools.

Pico SDK 2.1.1: SDK oficial da Raspberry Pi Foundation com APIs de hardware e suporte a FreeRTOS.

CMake ≥ 3.13: Sistema de build multiplataforma para gerenciamento de dependências e compilação.

Ninja 1.12.1: Build system focado em velocidade, backend do CMake.

GCC ARM 14.2: Compilador C/C++ para arquitetura ARM Cortex-M0+.

picotool 2.1.1: Utilitário oficial para upload de firmware e interação com o Pico.

Git: Sistema de controle de versão distribuído.

Minicom: Terminal serial para debug e visualização de logs.

1.3.3. Bibliotecas Externas

FreeRTOS Kernel: Sistema operacional de tempo real open-source, amplamente utilizado em sistemas embarcados. Versão: Compatível com Pico SDK 2.1.1 (FreeRTOS Kernel 10.4.3+)

LVGL 8.3.11: Light and Versatile Graphics Library - biblioteca gráfica para interfaces embarcadas.

Repositório: https://github.com/lvgl/lvgl

Mongoose: Biblioteca de rede embarcada para servidores web e clientes HTTP/HTTPS.

Repositório: https://github.com/cesanta/mongoose

cJSON: Parser JSON leve escrito em C puro, ideal para sistemas embarcados.

Repositório: https://github.com/DaveGamble/cJSON

FatFS (no-OS-FatFS-SD-SPI-RPi-Pico): Sistema de arquivos FAT32 para cartões SD via SPI, portado para Raspberry Pi Pico.

Repositório: https://github.com/carlk3/no-OS-FatFS-SD-SPI-RPi-Pico

1.4. Especificação de Requisitos

1.4.1. Requisitos Funcionais

Os requisitos funcionais descrevem as funcionalidades que o sistema deve oferecer, ou seja, o que o sistema deve ser capaz de fazer para atender às necessidades dos usuários e da clínica.

ID	Requisito
RF01	Conectar à rede WiFi e sincronizar horário via NTP
RF02	Baixar agendamentos do dia via API REST
RF03	Armazenar dados localmente em cartão SD
RF04	Ler QR Code apresentado pelo paciente
RF05	Validar agendamento (ID, CPF, horário)
RF06	Cadastrar e verificar digital do paciente
RF07	Enviar dados biométricos para o servidor
RF08	Exibir informações da consulta na tela
RF09	Mostrar mensagens de erro quando necessário
RF10	Sincronizar dados de check-in com o servidor
RF11	Operar com dados locais minimizando necessidade de conexão

1.4.2. Requisitos Não-Funcionais

Os requisitos não-funcionais especificam critérios de qualidade, desempenho e restrições técnicas que o sistema deve atender, definindo como as funcionalidades devem ser entregues.

ID	Requisito
RNF01	Tempo de resposta inferior a 3 segundos
RNF02	Interface gráfica sem travamentos
RNF03	Recuperação automática de erros
RNF04	Reinício automático em caso de falha crítica
RNF05	Armazenamento local para pelo menos 1 dia de operação
RNF06	Interface simples e intuitiva
RNF07	Operação contínua sem supervisão

1.5. Arquitetura do Sistema

1.5.1. Arquitetura de Hardware

O sistema é composto por um microcontrolador central (Raspberry Pi Pico W) conectado a diversos periféricos através de interfaces de comunicação serial.

Componentes

Componente	Modelo	Interface	Função
Microcontrolador	RP2040W	—	Processamento central
Display TFT	4.0" 480×320 ST7796	SPI0	Exibição da interface
Touch Capacitivo	FT6336U	I2C1	Entrada do usuário
Leitor QR Code	GM67	UART0	Leitura de QR Codes
Leitor Biométrico	R307S	UART1	Impressão digital (futuro)
Cartão SD	Módulo SPI	SPI1	Armazenamento local

Pinagem Principal

Periférico	Pinos
Display (SPI0)	MOSI: GP19, SCK: GP18, CS: GP17, DC: GP20, RST: GP21

Touch (I2C1)	SDA: GP6, SCL: GP7, RST: GP8, INT: GP9
QR Reader (UART0)	TX: GP0, RX: GP1
Fingerprint (UART1)	TX: GP4, RX: GP5
SD Card (SPI1)	MOSI: GP15, MISO: GP12, SCK: GP14, CS: GP13

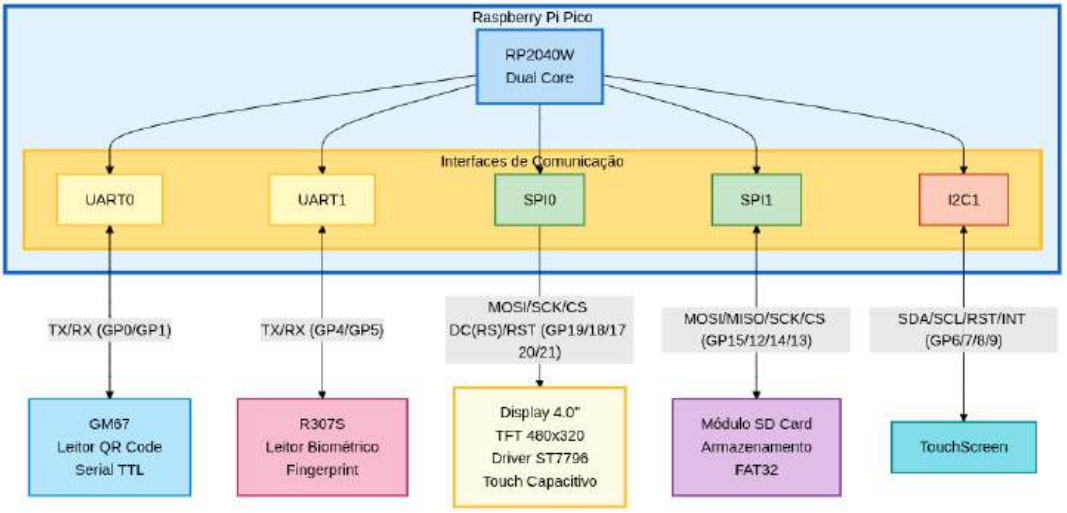


Figura 1: Diagrama de Hardware — ver diagrama hardware_diagram.md

1.5.2. Arquitetura de Software

O software foi projetado em uma arquitetura de 5 camadas, promovendo separação de responsabilidades e baixo acoplamento:

Camada	Responsabilidade	Exemplos
Application	Fluxo da aplicação, UI, orquestração	Tasks FreeRTOS, StateMachine, Screens
Service	Lógica de negócio	AppointmentService, DataStorageService, TimeService, NetworkSyncService, FingerprintService
Adapter	Integração com bibliotecas externas	LVGLDisplayAdapter, LVGLTouchAdapter
Driver	Comunicação com periféricos	ST7796Driver, FT6336U_Driver, GM67_Driver, R307S_Driver
HAL	Abstração de hardware	HAL_SPI_RP2040, HAL_I2C_RP2040, HAL_UART_RP2040

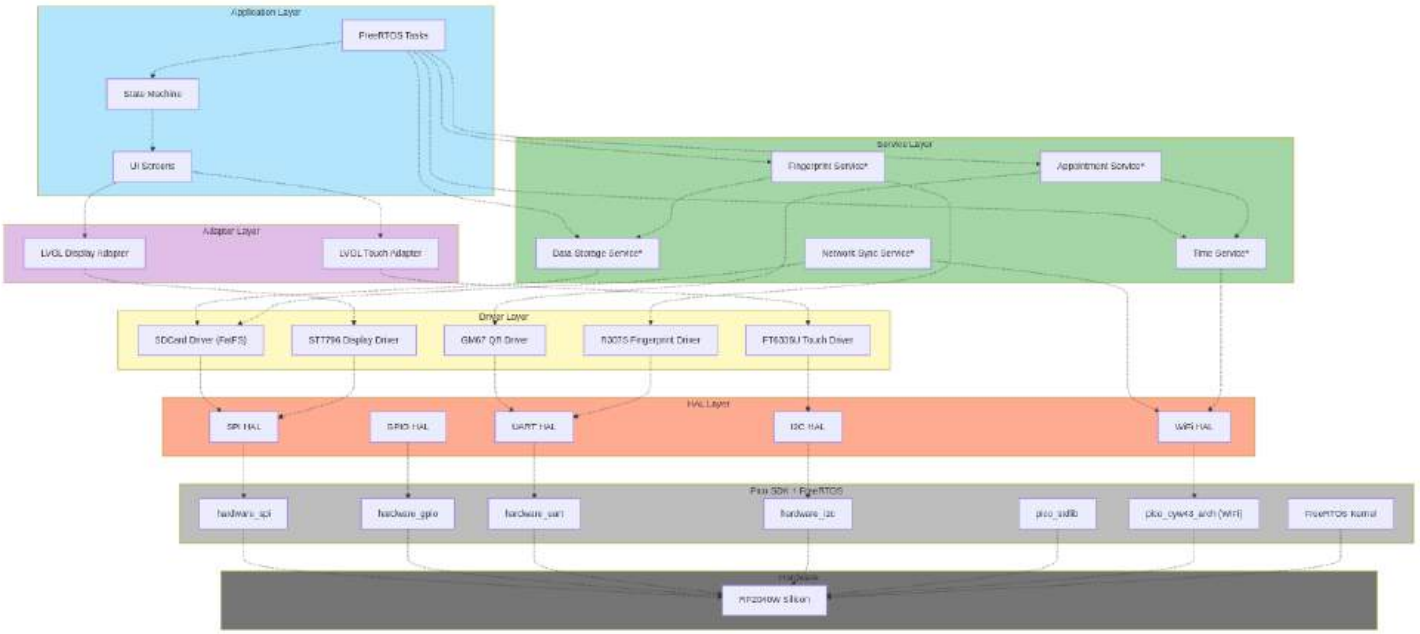


Figura 2: Diagrama de Arquitetura de Software em Camadas — ver diagrama software_architecture_layers_diagram.md

1.6. Implementação

1.6.1. Estrutura do Projeto

O projeto segue uma organização modular que separa código-fonte, configurações, bibliotecas externas e documentação:

```
├── CMakeLists.txt
├── pico_sdk_import.cmake
│
├── config/
│   ├── api_config.hpp                # URL da API, endpoints, credenciais, timeouts
│   ├── display_config.hpp            # Resolução, rotação, SPI speed, buffer LVGL
│   ├── fingerprint_config.hpp        # UART e baudrate do sensor biométrico
│   ├── FreeRTOSConfig.h              # Configuração do kernel FreeRTOS
│   ├── hw_config.c                   # Configuração do SD Card (FatFS)
│   ├── lv_conf.h                     # Configuração do LVGL
│   ├── lvgl_tick_custom.h            # Tick customizado do LVGL para FreeRTOS
│   ├── lwipopts.h                    # Configuração do lwIP (TCP/IP stack)
│   ├── mongoose_config.h             # Configuração do Mongoose HTTP
│   ├── pin_config.hpp                # Mapeamento de todos os GPIOs
│   ├── qr_code_reader_config.hpp     # UART e baudrate do scanner QR
│   ├── sntp_callbacks.h              # Callbacks de sincronização NTP
│   └── wifi_config.hpp               # SSID, senha e país do WiFi
│
├── src/
│   ├── main.cpp                     # Entry point: init drivers → create tasks →
scheduler
│   │
│   ├── core/
│   │   ├── state_machine.hpp         # Máquina de estados (singleton, 11 estados)
│   │   └── state_machine.cpp
```

```
│   │
│   ├── hal/
│   │   ├── interfaces/
│   │   │   ├── hal_spi_interface.hpp   # Interface abstrata SPI
│   │   │   ├── hal_i2c_interface.hpp   # Interface abstrata I2C
│   │   │   ├── hal_uart_interface.hpp  # Interface abstrata UART
│   │   │   └── hal_wifi_interface.hpp  # Interface abstrata WiFi
│   │   └── rp2040/
│   │       ├── hal_spi_rp2040.hpp/.cpp
│   │       ├── hal_i2c_rp2040.hpp/.cpp
│   │       ├── hal_uart_rp2040.hpp/.cpp
│   │       └── hal_wifi_rp2040.hpp/.cpp
│   │
│   ├── drivers/
│   │   ├── display/
│   │   │   ├── interface/display_interface.hpp
│   │   │   └── st7796/st7796_driver.hpp/.cpp
│   │   ├── touch/
│   │   │   ├── interface/touch_interface.hpp
│   │   │   └── ft6336u/ft6336u_driver.hpp/.cpp
│   │   ├── qr_code_reader/
│   │   │   ├── interface/qr_interface.hpp
│   │   │   └── gm67/gm67_driver.hpp/.cpp
│   │   └── fingerprint/
│   │       ├── interface/fingerprint_interface.hpp
│   │       └── r307s/r307s_driver.hpp/.cpp
│   │
│   ├── adapters/lvgl/
│   │   ├── lvgl_display_adapter.hpp/.cpp
│   │   └── lvgl_touch_adapter.hpp/.cpp
│   │
│   ├── services/
│   │   ├── appointment_service.hpp/.cpp     # Validação de QR e agendamentos
│   │   ├── data_storage_service.hpp/.cpp    # Leitura/escrita no SD Card
│   │   ├── fingerprint_service.hpp/.cpp     # Cadastro e verificação biométrica
│   │   ├── network_sync_service.hpp/.cpp    # Cliente HTTP (Mongoose)
│   │   └── time_service.hpp/.cpp            # NTP → RTC, data/hora
│   │
│   ├── tasks/
│   │   ├── task_wifi_time_init.hpp/.cpp     # Conexão WiFi + sincronização NTP
│   │   ├── task_lvgl.hpp/.cpp               # Loop LVGL + gerência de telas
│   │   ├── task_qr_reader.hpp/.cpp          # Leitura contínua do scanner QR
│   │   ├── task_qr_validator.hpp/.cpp       # Validação e orquestração do check-in
│   │   └── task_network.hpp/.cpp            # Download de agendamentos e templates
│   │
│   ├── ui/screens/
│   │   ├── welcome_screen.hpp/.cpp          # Tela principal (idle)
│   │   ├── init_status_screen.hpp/.cpp      # Tela de boot (WiFi, NTP, SD)
│   │   ├── appointment_screen.hpp/.cpp      # Check-in confirmado
│   │   ├── error_screen.hpp/.cpp            # Erro recuperável
│   │   ├── error_critical_screen.hpp/.cpp   # Erro crítico + countdown reboot
│   │   ├── fingerprint_operation_screen.hpp/.cpp  # Verificação/cadastro digital
│   │   └── fingerprint_upload_screen.hpp/.cpp     # Upload de template
│   │
│   └── tests/code_examples/
│       ├── README.md                        # Instruções dos demos
│       ├── lvgl_rtos_demo.cpp               # Demo display + touch
│       ├── qr_code_reader_demo.cpp          # Demo scanner QR
│       ├── fingerprint_demo.cpp             # Demo sensor biométrico
│       ├── sdcard_fatfs_demo.cpp            # Demo SD Card
│       └── wifi_demo.cpp                    # Demo WiFi
│
├── lib/
│   ├── FreeRTOS-Kernel/                     # Submodule
│   └── no-OS-FatFS/                         # Submodule
│
```

```
└── docs/
    ├── dev/                                # Notas de desenvolvimento por periférico
    └── diagrams/
        ├── hardware_diagram.md
        ├── software_architecture_layers_diagram.md
        ├── class_diagrams.md
        ├── sequence_diagrams.md
        └── state_machine_diagram.md
```

Observações importantes:

- **Bibliotecas externas:** LVGL, Mongoose e cJSON são baixadas automaticamente pelo CMake via FetchContent. FreeRTOS e FatFS são submodules Git em lib/.
- **Build:** A pasta build/ é gerada automaticamente e não é versionada (incluída no .gitignore).
- **Configurações:** Todas as configurações de hardware e rede estão centralizadas em config/ para facilitar adaptação a diferentes ambientes.

Repositório do projeto desenvolvido:
[AdryRocha/medical-clinic-checkin: Repositório com o desenvolvimento de código embarcado para funcionalidades de check-in em clínicas médicas.]

1.6.2. Controle de Versão

O projeto utiliza Git com estratégia de branching baseada em GitFlow simplificado para garantir código estável e facilitar colaboração.

Estrutura de Branches

main: Branch principal de produção contendo apenas código estável e testado. Recebe commits exclusivamente da branch dev via merge, e cada merge recebe uma tag de versão semântica (ex: v1.0.0, v1.1.0). Mantém histórico linear e limpo para rastreamento de releases.

dev: Branch de integração e desenvolvimento contínuo onde todas as features são integradas e testadas. Testes manuais em hardware são realizados aqui antes de mesclar na main. Serve como ponto de partida para criar novas branches feature/* e deve estar sempre sincronizada e em estado funcional.

feature/*: Branches de desenvolvimento isolado de funcionalidades específicas com nomenclatura descritiva (ex: feature/hal-drivers, feature/qr-validation, feature/biometric-auth). Código novo é escrito nestas branches sem afetar dev e são mescladas na dev via Pull Request após revisão de código.

1.7. Testes e Validação

1.7.1. Estratégia de Testes

O projeto adotou uma estratégia de testes bottom-up, validando cada componente isoladamente antes da integração.

1.7.2. Demos Standalone

Foram desenvolvidos demos standalone para validação individual de cada componente de hardware:

Demo	Arquivo	Componente Testado
LVGL RTOS Demo	lvgl_rtos_demo.cpp	Display TFT + Touch + FreeRTOS + Interface gráfica
QR Code Reader Demo	qr_code_reader_demo.cpp	Scanner GM67 via UART, decodificação de dados
SD Card FatFS Demo	sdcard_fatfs_demo.cpp	Leitura/escrita de arquivos, sistema FAT32
WiFi Demo	wifi_demo.cpp	Conexão WiFi, requisições HTTP, NTP
Fingerprint Demo	fingerprint_demo.cpp	Sensor R307S (em desenvolvimento)

Localização: src/tests/code_examples/

Instruções de execução: Ver src/tests/code_examples/README.md

1.7.3. Testes Integrados

Após validação individual, foram realizados testes integrados do fluxo completo:

1. Teste de inicialização: Verificação de conexão WiFi, sincronização NTP e montagem do SD Card
2. Teste de download: Download de agendamentos da API de persistência local
3. Teste de validação: Leitura de QR Codes válidos e inválidos
4. Teste de atualização: Marcação de check-in e sincronização com API
5. Teste de recuperação: Simulação de erros e verificação de transições de estado

1.8. Resultados

1.8.1. Funcionalidades Implementadas

O sistema implementa com sucesso um fluxo de check-in automatizado, integrando hardware (display touchscreen ST7796, scanner QR GM67, leitor biométrico R503 e cartão SD) e software (FreeRTOS, LVGL, drivers customizados HAL) de forma estável.

Conectividade e Sincronização: O sistema conecta-se automaticamente à rede WiFi na inicialização, sincroniza horário via NTP para validação temporal precisa, autentica na API REST e baixa agendamentos com persistência local em cartão SD (formato JSON).

Validação de Check-in: A leitura e validação de QR Code ocorre de forma fluida, verificando ID, CPF, hash, horário (janela de ±30 minutos) e status do agendamento. O sistema impede check-ins duplicados e fornece feedback claro para todas as situações.

Autenticação Biométrica: Implementa cadastro e verificação de digitais através do sensor R503, com armazenamento local e sincronização com o servidor. O sistema identifica automaticamente se o paciente precisa cadastrar digital ou apenas verificá-la, enviando os templates biométricos para a API após captura.

Operação Resiliente: Recupera-se automaticamente de erros não-críticos, retorna à tela inicial após timeouts configurados e mantém operação estável mesmo com falhas temporárias de conectividade ou hardware.

1.8.2. Nível de Maturidade Tecnológica (TRL)

A escala TRL (Technology Readiness Level) é uma metodologia desenvolvida pela NASA e amplamente adotada para avaliar o grau de maturidade de uma tecnologia. A escala possui 9 níveis:

- TRL 1-3: Pesquisa básica (princípios observados, conceito formulado, prova de conceito experimental)
- TRL 4-6: Desenvolvimento tecnológico (validação em laboratório, validação em ambiente relevante, demonstração em ambiente relevante)
- TRL 7-9: Demonstração e implementação (protótipo em ambiente operacional, sistema qualificado, sistema comprovado em operação)

TRL Atual: Nível 4 - Validação de componentes e sistema integrado em ambiente de laboratório.

O projeto encontra-se no TRL 4 com base nas seguintes evidências.

Características que comprovam TRL 4:

- ✅ Sistema integrado completo testado em bancada de laboratório
- ✅ Hardware montado em placa de prototipagem soldada com conectores crimpados
- ✅ Todos os componentes validados individualmente e em conjunto
- ✅ Comunicação WiFi e sincronização via API REST funcionais
- ✅ Autenticação biométrica implementada e operacional
- ✅ Fluxo completo de check-in (QR Code → Validação → Biometria → Confirmação) funcional
- ✅ Sincronização de dados em tempo real com servidor
- ✅ Armazenamento local em SD Card com formato JSON
- ✅ Arquitetura de software modular e documentada
- ✅ Todos os requisitos funcionais e não-funcionais atendidos

O que falta para TRL 5 (Validação em ambiente relevante):

- ❌ Ausência de PCB customizado (placa atual é de prototipagem soldada)
- ❌ Ausência de case profissional para instalação em recepção
- ❌ Hardware não preparado para ambiente clínico real
- ❌ Comunicação HTTP sem criptografia (impede uso com dados reais)
- ❌ Ausência de validação completa de hash do QR Code
- ❌ Dados no SD Card sem criptografia
- ❌ Sistema de autenticação básico (impossibilita testes com usuários reais)
- ❌ Backend não está em ambiente de produção
- ❌ Sem monitoramento e telemetria de operação
- ❌ Sem sistema de atualização remota (OTA)
- ❌ Certificações regulatórias (ANATEL, INMETRO) não obtidas

O que é necessário para TRL 5:

Para atingir TRL 5, o sistema precisa ser validado em ambiente que simule operação real de clínica, o que exige: PCB dedicado, case profissional, implementação completa de segurança (HTTPS, criptografia), sistema robusto de autenticação e infraestrutura de backend em produção.

2. AGENDAMENTOS DE CONSULTAS POR WEB API E BOT DO TELEGRAM

2.1. INTRODUÇÃO

O presente documento descreve a arquitetura e funcionalidades do Sistema de Gerenciamento de Clínicas Médicas, composto por uma API REST e um Bot Telegram para agendamento de consultas. Este sistema foi desenvolvido para integração com dispositivo embarcado responsável pela autenticação biométrica e check-in de pacientes.

O sistema oferece uma solução completa para gestão de clínicas médicas, abrangendo desde o cadastro de pacientes e profissionais até o agendamento de consultas e validação de presença através de QR Code. A integração com sistema embarcado permitirá a autenticação por biometria digital, proporcionando maior segurança e agilidade no processo de check-in.

Repositório do grupo em github privado com o desenvolvimento da API:
https://github.com/Luana-Menezes/Clinicas_Medicas_WebAPI

2.2. ARQUITETURA DO SISTEMA

2.2.1. Componentes Principais

O sistema é dividido em dois componentes principais que trabalham de forma integrada:

a) API REST (FastAPI): Backend completo desenvolvido em Python utilizando o framework FastAPI, responsável por toda a lógica de negócio e persistência de dados.

b) Bot Telegram: Interface conversacional que permite aos pacientes agendar consultas e gerenciar suas informações através do aplicativo Telegram.

2.2.2. Stack Tecnológico

API REST: Desenvolvida em Python 3.12+ com FastAPI 0.104.1, utiliza SQLAlchemy 2.0.23 como ORM para PostgreSQL, Pydantic 2.5.0 para validação de dados, JWT para autenticação e Segno 1.6.1 para geração de QR Codes.

Bot Telegram: Implementado em Python 3.12+ com a biblioteca python-telegram-bot, realizando integração via HTTP com a API REST para envio de QR Codes e notificações aos pacientes.

2.3. API REST

2.3.1. Visão Geral

A API REST constitui o núcleo do sistema, fornecendo endpoints para todas as operações de cadastro, consulta, agendamento e geração de QR Code. A API foi desenvolvida seguindo os princípios REST e oferece documentação interativa através do Swagger UI.

- URL Base: http://localhost:8000
- Documentação: http://localhost:8000/docs

2.3.2. Autenticação e Segurança

O sistema utiliza autenticação baseada em tokens JWT (JSON Web Tokens). Todas as rotas da API, exceto a rota de autenticação e geração de QR Code, requerem um token válido no cabeçalho da requisição.

Formato do Token: Authorization: Bearer <token>. Tempo de Expiração: Configurável via variável de ambiente Algoritmo: HS256

2.3.3. Estrutura de Dados

2.3.3.1. Pacientes

Entidade responsável por armazenar informações dos pacientes cadastrados no sistema.
Campos:

- id: Integer (Primary Key)
- nome: String(200) - Nome completo do paciente
- cpf: String(11) - CPF único do paciente (apenas dígitos)
- aceita_digital: Boolean - Indicador de aceitação de coleta biométrica
- fingerprint_data: LargeBinary - Dados da impressão digital (opcional)

Relacionamentos:

- Um paciente pode ter múltiplas consultas

2.3.3.2. Categorias

Representa as especialidades médicas disponíveis na clínica.

Campos:

- id: Integer (Primary Key)
- nome: String(100) - Nome da especialidade

Exemplos: Cardiologia, Dermatologia, Ortopedia

2.3.3.3. Profissionais

Cadastro dos profissionais de saúde que atendem na clínica.

Campos:

- id: Integer (Primary Key)
- nome: String(200) - Nome completo do profissional
- categoria_id: Integer (Foreign Key) - Especialidade do profissional

Relacionamentos:

- Um profissional pertence a uma categoria
- Um profissional pode ter múltiplos horários de atendimento
- Um profissional pode ter múltiplas consultas

2.3.3.4. Horários Profissionais

Define os horários de disponibilidade dos profissionais por dia da semana.

Campos:

- id: Integer (Primary Key)
- profissional_id: Integer (Foreign Key)
- dia_semana: Integer - Dia da semana (0=Segunda, 6=Domingo)
- hora_inicio: String(5) - Horário inicial (formato HH:MM)
- hora_fim: String(5) - Horário final (formato HH:MM)
- duracao_minutos: Integer - Duração de cada consulta em minutos

2.3.3.5. Consultas

Registro dos agendamentos de consultas realizados.

Campos:

- id: Integer (Primary Key)
- paciente_id: Integer (Foreign Key)
- profissional_id: Integer (Foreign Key)
- data: String(10) - Data da consulta (formato YYYY-MM-DD)
- horario: String(5) - Horário da consulta (formato HH:MM)
- status: String(20) - Status da consulta (agendada, realizada, cancelada)

Relacionamentos:

- Uma consulta pertence a um paciente
- Uma consulta pertence a um profissional

2.4. BOT TELEGRAM

2.4.1. Visão Geral

O Bot Telegram oferece uma interface conversacional intuitiva para que pacientes possam agendar consultas diretamente através do aplicativo Telegram. O bot guia o usuário através de um fluxo estruturado, coletando todas as informações necessárias para o agendamento.

2.4.2. Funcionalidades

2.4.2.1. Comandos Disponíveis

/start: Inicia interação com o bot e exibe menu principal

Opções:

- Agendar Consulta
- Recuperar Consulta

/agendar: Inicia fluxo de agendamento de consulta

/recuperar: Recupera informações de consultas agendadas

/ajuda: Exibe informações de ajuda e comandos disponíveis

2.4.3. Fluxo de Agendamento

O processo de agendamento segue uma sequência estruturada de estados:

Estado 1: Coleta de CPF

- Bot solicita CPF do paciente
- Validação de formato (11 dígitos numéricos)
- Verificação no banco de dados via API

Estado 2: Cadastro (se necessário)

- Se CPF não encontrado, solicita nome completo
- Pergunta sobre aceitação de coleta biométrica
- Cadastra novo paciente via API

Estado 3: Seleção de Especialidade

- Exibe lista de especialidades disponíveis
- Apresenta opções em formato de teclado customizado

Estado 4: Seleção de Profissional

- Lista profissionais da especialidade escolhida
- Exibe nome dos profissionais disponíveis

Estado 5: Seleção de Data

- Apresenta próximos dias disponíveis
- Utiliza teclado com dias da semana

Estado 6: Seleção de Horário

- Busca horários disponíveis via API
- Exibe apenas horários livres

Estado 7: Confirmação

- Exibe resumo completo do agendamento
- Solicita confirmação do paciente

- Opções: Confirmar ou Cancelar

Estado 8: Finalização

- Registra consulta via API
- Gera QR Code para check-in
- Envia QR Code ao paciente
- Fornece instruções sobre o check-in

3. INTEGRAÇÃO WEB API COM SISTEMA EMBARCADO

3.1. Visão Geral da Integração

O sistema Web API e Bot Telegram foi desenvolvido considerando integração com dispositivo embarcado para autenticação biométrica e check-in automatizado. Esta integração permitirá que pacientes realizem check-in de forma autônoma através de leitura de QR Code e validação biométrica.

3.2. Arquitetura de Integração

O dispositivo embarcado atuará como cliente da API REST, realizando as seguintes operações:

a) Leitura de QR Code: O embarcado lê o QR Code gerado pelo sistema, extraindo informações da consulta (ID, CPF, nome, hash).

b) Validação de Consulta: Na inicialização, o embarcado baixa todos os agendamentos do dia através da API e armazena em formato JSON no cartão SD. A validação verifica ID, CPF, hash do QR Code, janela de tempo (±30 minutos) e status do agendamento, impedindo check-ins duplicados.

c) Coleta Biométrica: Para pacientes que aceitam biometria, o sistema realiza duas operações: cadastro de nova digital (enrollment) caso não possua, ou verificação de digital já cadastrada.

d) Autenticação Biométrica: As digitais são armazenadas no SD Card e no sensor R307S e comparadas internamente.

e) Upload de Biometria: Após cadastro de nova digital, o sistema envia os templates biométricos em formato base64 para a API, permitindo backup e sincronização com o banco de dados.

f) Sincronização de Check-in: O NetworkSyncService sincroniza em tempo real o status de check-in e dados biométricos com a API. O sistema opera também em modo offline, salvando dados localmente quando sem conexão e sincronizando posteriormente.

4. PROTÓTIPO FÍSICO

O protótipo físico do sistema foi montado em uma caixa com todos os módulos fixados e interligados por uma placa de prototipagem soldada. As ligações entre os módulos foram feitas com solda estanho, garantindo

conexões mecânica e eletricamente firmes, reduzindo bastante o risco de falhas por mau contato típico de jumpers soltos.

Para interligação dos cabos foram utilizados conectores JHT (dois conectores de 4 pinos e um de 6 pinos) e um conector tipo KK para o flat-cable do display, todos crimpados/montados manualmente. Esses conectores permitem conectar e desconectar rapidamente cada componente (display, leitor de QR code, leitor biométrico, etc.), facilitando manutenção e futuras alterações sem necessidade de ressoldar nada.

A pinagem foi organizada em barramentos lógicos, separando alimentação (5 V e 3,3 V), terra (GND) e sinais de comunicação. Os sinais seriais dos periféricos (por exemplo, UART do leitor biométrico e do leitor de QR code) foram roteados em pares TX/RX dedicados, enquanto o display e o cartão SD utilizam linhas específicas de dados e controle (como clock, MOSI/MISO ou pinos de comando), sempre tentando manter a mesma cor de fio para cada função em todas as conexões. Essa padronização facilita o rastreamento de cada sinal na placa, simplifica o diagnóstico de falhas e reduz a chance de inversão de pinos durante a montagem ou manutenção.

Como resultado, a organização interna do protótipo ficou bem estruturada, com boa acessibilidade aos módulos e melhor confiabilidade elétrica, como ilustrado nas imagens no apêndice.

5. TRABALHOS FUTUROS

O desenvolvimento futuro do sistema requer melhorias em diferentes frentes para atingir maturidade tecnológica adequada para implantação em ambiente clínico real:

Item	Descrição	Categoria
Case 3D profissional	Desenvolvimento de estrutura física para instalação em balcões de recepção	Hardware
PCB customizado	Substituição das conexões em protoboard por placa de circuito impresso dedicada	Hardware
Upgrade para RP2350	Migração para Raspberry Pi Pico 2 (RP2350) para maior desempenho e recursos avançados	Hardware
Sistema de logs	Implementação de logging estruturado no SD Card com rotação automática	Software
Watchdog e recuperação	Watchdog por hardware para reinicialização automática e crash logs	Software
Melhorias de Segurança	Features de segurança avançadas no embarcado e API não implementadas por limitação de tempo: validação de hash do QR Code, criptografia de dados SD Card e comunicação HTTPS no embarcado; sistema robusto de usuários, criptografia de dados sensíveis no banco, etc.	Segurança
Backend em produção	Deploy em servidor dedicado com alta disponibilidade e redundância	Infraestrutura

Atualização OTA	Sistema de atualização remota de firmware via rede	Infraestrutura
Testes piloto em clínica	Validação em ambiente real com usuários finais e ajustes de UX	Validação
Certificações e Conformidade	ANATEL, INMETRO e conformidade com LGPD	Validação

6. CONCLUSÃO

Este trabalho apresentou o desenvolvimento de um sistema embarcado de check-in para clínicas médicas utilizando o microcontrolador Raspberry Pi Pico W, demonstrando a viabilidade de soluções de autoatendimento de baixo custo para o setor de saúde.

A arquitetura em camadas (HAL, Drivers, Serviços e Aplicação) promoveu modularidade e facilitou o desenvolvimento incremental. A separação clara de responsabilidades permite manutenção simplificada e extensão futura do sistema sem reestruturação significativa.

O uso de FreeRTOS permitiu uma aplicação multitarefa responsiva, onde diferentes componentes (leitura de QR, validação biométrica, interface gráfica, comunicação de rede) operam de forma concorrente e coordenada. A biblioteca LVGL proporcionou interface gráfica intuitiva e adequada ao contexto de autoatendimento.

A integração com API REST, autenticação biométrica R503 e armazenamento local em SD Card garantem flexibilidade operacional, permitindo funcionamento em modo online (sincronização em tempo real) e offline (dados previamente baixados).

O sistema implementado atende aos requisitos funcionais e não-funcionais estabelecidos, incluindo validação de QR Code, verificação biométrica, sincronização de dados e operação resiliente. O projeto demonstra como tecnologias embarcadas modernas podem resolver problemas práticos, contribuindo para eficiência operacional e melhoria da experiência do usuário em ambientes de saúde.

7. APÊNDICE A: Manual do Usuário

7.1. Manual do usuário do bot de agendamento

7.1.1. Sobre o Bot

O Bot do Telegram permite que pacientes realizem as seguintes ações de forma rápida e simples:

- Agendar consultas médicas
- Recuperar informações de consultas já agendadas
- Receber o QR Code para realização do check-in no totem da clínica

O atendimento é realizado diretamente pelo aplicativo Telegram, sem necessidade de ligação telefônica ou atendimento presencial.

7.1.2. Como Iniciar

1. Abra o aplicativo Telegram em seu celular ou computador.
2. Pesquise pelo nome do bot da clínica @ClinicaMedicaAgendamentos_bot
3. Toque no bot para abrir a conversa.
4. Digite o comando /iniciar.
5. Escolha uma das opções exibidas no menu /agendar /minhas_consultas ou /ajuda.

7.1.3. Como Agendar uma Consulta

Passo 1 – Informar CPF

- O bot solicitará seu CPF (apenas números).
- O CPF é utilizado para identificar seu cadastro.

Passo 2 – Cadastro (se necessário)

Se o CPF não estiver cadastrado, o bot solicitará:

- Nome completo
- Confirmação sobre aceitação de biometria digital

Após isso, o cadastro será realizado automaticamente.

Passo 3 – Escolher Especialidade

O bot exibirá as especialidades médicas disponíveis.

Selecione a desejada.

Passo 4 – Escolher Profissional

Será exibida a lista de profissionais da especialidade escolhida.

Selecione o profissional desejado.

Passo 5 – Escolher Data

O bot apresentará as datas disponíveis para atendimento.

Selecione o dia desejado.

Passo 6 – Escolher Horário

Serão exibidos os horários disponíveis.

Selecione o horário mais adequado.

Passo 7 – Confirmar Agendamento

O bot exibirá um resumo completo da consulta:

- Nome do profissional
- Data
- Horário

Confirme para finalizar o agendamento.

Passo 8 – Receber QR Code

Após a confirmação, o bot enviará um QR Code.

Esse código deverá ser apresentado no totem da clínica no dia da consulta para realizar o check-in.

7.1.4. Como Recuperar o QR Code de uma Consulta

Caso você tenha perdido a mensagem com o QR Code ou precise acessá-lo novamente, é possível recuperá-lo diretamente pelo bot.

Passo 1 – Acessar o Menu

Abra a conversa com o bot no Telegram.

Digite /iniciar e depois selecione uma opção correspondente /minhas_consultas

Passo 2 – Informar CPF

O bot solicitará novamente seu CPF para localizar suas consultas agendadas.

Passo 3 – Selecionar a Consulta

Se houver mais de uma consulta agendada, o bot exibirá a lista com:

- Profissional
- Data
- Horário

Selecione a consulta desejada.

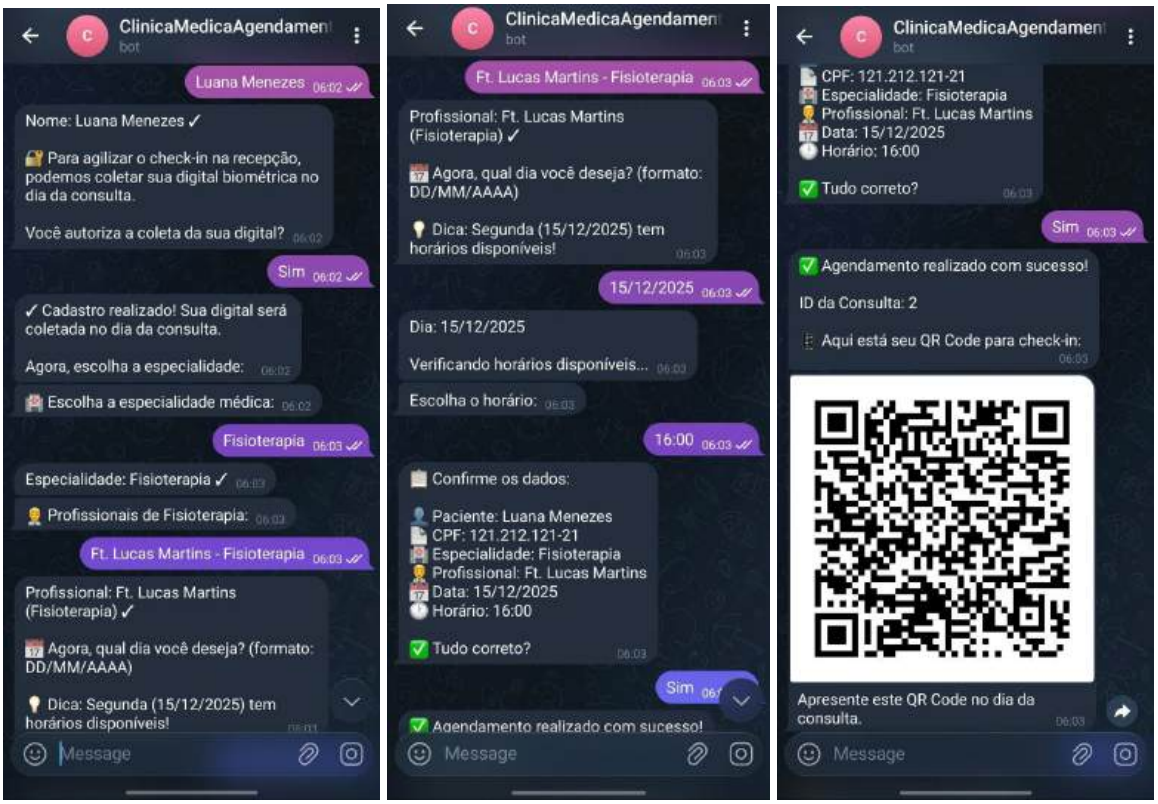
Passo 4 – Receber o QR Code

Após a seleção, o bot reenviará o QR Code correspondente à consulta escolhida.
O QR Code poderá ser utilizado normalmente no totem para realizar o check-in.

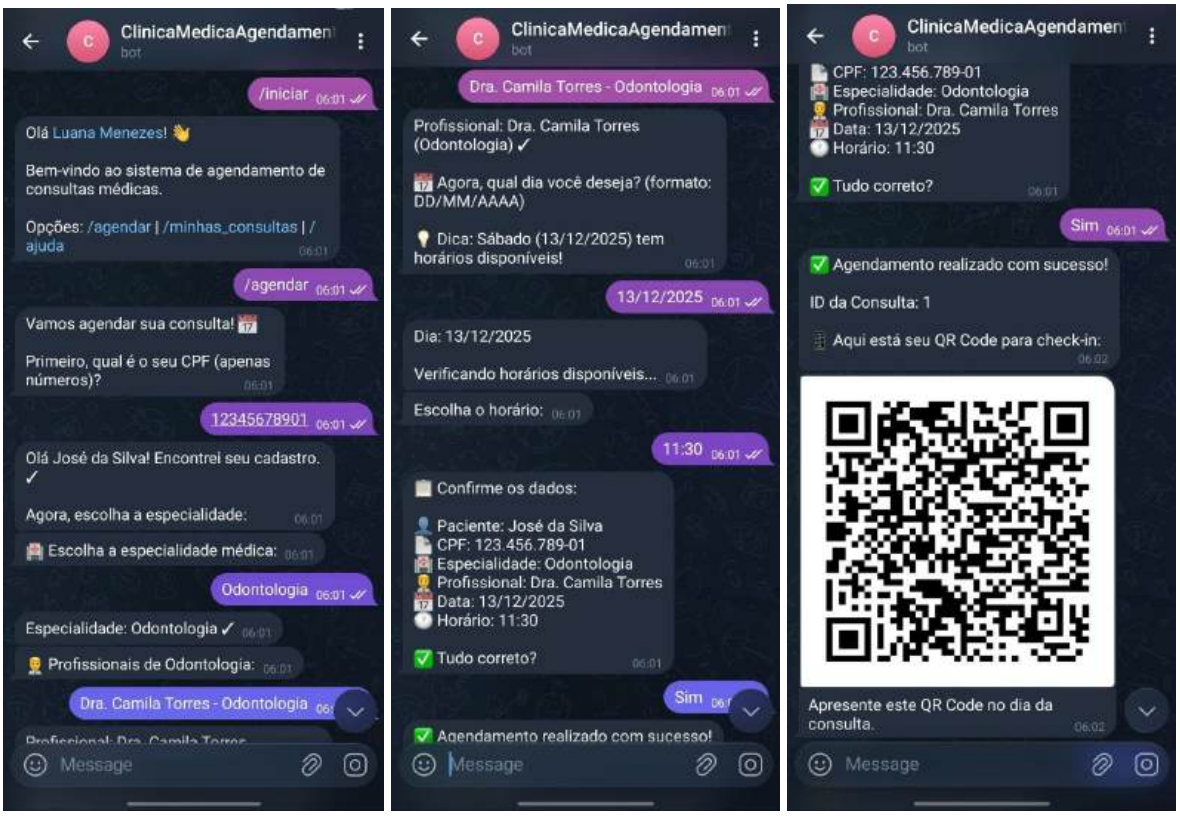
7.1.5. Capturas de Tela do Agendamento por Telegram

Segue abaixo exemplos de print screen do BOT do Telegram:

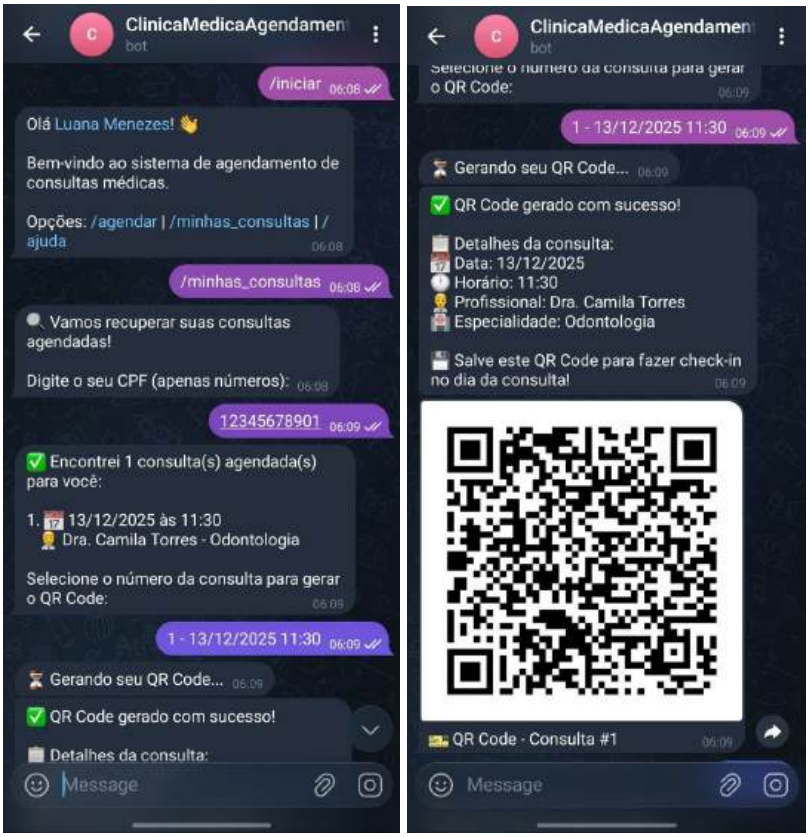
Agendamento usuário novo



Agendamento usuário já existente



Consultar consultas agendados e recuperação do QR Code

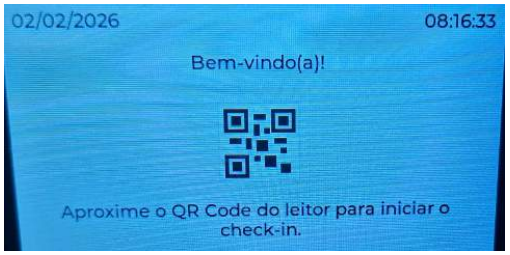


7.2. Manual do usuário Totem de Check-in (Sistema Embarcado)

O totem confirma a presença do paciente com base nas informações do agendamento e, quando aplicável, na biometria.

1. Leitura do QR Code

- Na tela inicial, aproxime o QR Code do leitor.
- Aguarde a leitura automática.



Após a leitura, o sistema validará:

- Existência do agendamento
- Data e horário
- Situação do check-in

Se estiver tudo correto, o sistema seguirá para um dos dois cenários abaixo.

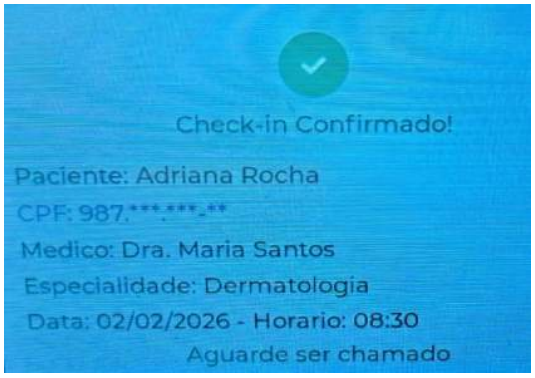
2. Cenários de Sucesso

2.1 Paciente NÃO optou por biometria

Se o paciente não autorizou o uso de impressão digital no momento do cadastro, o processo será concluído apenas com a validação do QR Code.

A tela exibirá:

- Nome do paciente
- Nome do profissional
- Data e horário da consulta
- Mensagem de confirmação



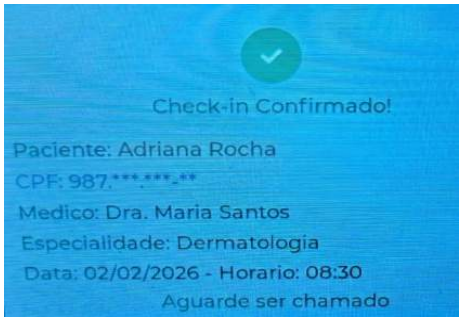
O check-in será registrado automaticamente. Nenhuma etapa adicional será necessária.

2.2 Paciente OPTOU por biometria

Possíveis situações:

a) Digital já cadastrada

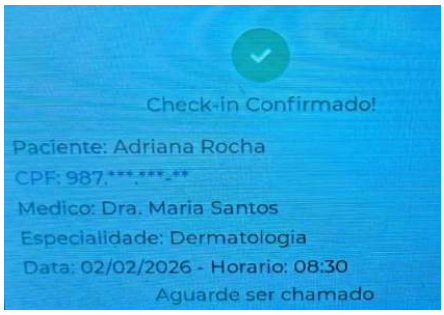
- Posicione o dedo no sensor.
- Aguarde a leitura.
- Após reconhecimento, o sistema exibirá a confirmação do check-in.



b) Digital ainda não cadastrada

Se for o primeiro uso após a autorização de biometria, o sistema solicitará o cadastro da digital.

- Posicione o dedo no sensor.
- Aguarde a leitura.
- Caso solicitado, repita o procedimento para confirmação.
- Após o cadastro bem-sucedido, o check-in será concluído automaticamente.



8. APÊNDICE B: Documentação Técnica

8.1. Protocolos de Comunicação

O sistema utiliza diversos protocolos de comunicação serial para interface com periféricos:

SPI (Serial Peripheral Interface)

Protocolo síncrono full-duplex utilizado para comunicação de alta velocidade. No projeto, é empregado para:

- Display TFT (ST7796): Transmissão de comandos e dados de vídeo;
- Cartão SD: Leitura e escrita de arquivos.

I2C (Inter-Integrated Circuit)

Protocolo síncrono half-duplex com endereçamento, adequado para periféricos de baixa velocidade. Utilizado para:

- Touchscreen capacitivo (FT6336U): Leitura de coordenadas de toque.

UART (Universal Asynchronous Receiver-Transmitter)

Protocolo assíncrono ponto-a-ponto utilizado para:

- Leitor de QR Code (GM67): Recepção de dados decodificados;

- Leitor biométrico (R307S): Comunicação com sensor de impressão digital

8.2. Detalhes de Implementação (HAL, Drivers, Serviços, Tasks)

8.2.1. Camada HAL (Hardware Abstraction Layer)

A camada HAL define interfaces abstratas para cada protocolo de comunicação, permitindo portabilidade do código.

Interface	Implementação RP2040	Métodos Principais
HAL_SPI_Interface	HAL_SPI_RP2040	init(), write(), read(), setCS(), setDC(), reset()
HAL_I2C_Interface	HAL_I2C_RP2040	init(), write(), read(), writeRead()
HAL_UART_Interface	HAL_UART_RP2040	init(), write(), read(), available(), writeByte(), readByte()

HAL_SPI_RP2040

A implementação do SPI para RP2040 encapsula as APIs do Pico SDK:

- Configuração de pinos: Define MOSI, MISO, SCK, CS, DC (Data/Command) e RST (Reset)
- Inicialização: Configura baudrate e modo SPI
- Controle de CS: Habilita/desabilita chip select para comunicação
- Controle de DC: Alterna entre envio de comandos e dados
- Reset de hardware: Pulso no pino RST para reinicialização do periférico

HAL_I2C_RP2040

A implementação do I2C para RP2040 encapsula as APIs do Pico SDK:

- Configuração de pinos: Define SDA (dados) e SCL (clock)
- Inicialização: Configura baudrate (geralmente 100 kHz ou 400 kHz)
- Comunicação com endereçamento: Suporta comunicação com múltiplos dispositivos via endereço I2C
- Operações combinadas: writeRead() permite escrever e ler em uma única transação

HAL_UART_RP2040

A implementação do UART para RP2040 encapsula as APIs do Pico SDK:

- Configuração de pinos: Define TX (transmissão) e RX (recepção)
- Inicialização: Configura baudrate (ex: 9600, 115200), bits de dados, paridade e stop bits
- Buffer de recepção: available() verifica dados disponíveis sem bloquear
- Transmissão assíncrona: Envio e recepção independentes sem sinal de clock

8.3. Camada de Drivers

Cada driver encapsula a lógica de comunicação com um periférico específico.

Driver	Periférico	Protocolo	Descrição
ST7796Driver	Display TFT	SPI	Inicialização, envio de comandos e pixels, controle de rotação
FT6336U_Driver	Touchscreen	I2C	Leitura de coordenadas e detecção de toques
GM67_Driver	Leitor QR	UART	Leitura de dados decodificados linha por linha
R307S_Driver	Biometria	UART	Captura e verificação de digitais (implementação futura)

ST7796Driver

Responsável pelo controle do display TFT de 4.0" com driver ST7796:

- Sequência de inicialização: Envia comandos de configuração (formato de cor, modo de scan, etc.)
- Envio de pixels: Escreve buffers de cor na memória do display
- Controle de rotação: Suporta 4 orientações (0°, 90°, 180°, 270°)
- Otimização: Utiliza DMA para transferências grandes de dados

FT6336U_Driver

Responsável pelo controle do touchscreen capacitivo FT6336U:

- Detecção de toque: Lê registradores I2C para identificar toques

- Coordenadas: Extrai posições X e Y do toque na tela
- Multi-touch: Suporta até 2 pontos de toque simultâneos
- Calibração: Ajusta coordenadas de acordo com a rotação do display

GM67_Driver

Responsável pela leitura do scanner de QR Code GM67:

- Recepção serial: Lê dados via UART conforme o scanner decodifica QR Codes
- Buffer de linha: Acumula caracteres até receber terminador '\n'
- Validação: Verifica se os dados recebidos são JSON válido
- Não-bloqueante: Opera de forma assíncrona sem travar a aplicação

8.4. Camada de Serviços

Os serviços implementam a lógica de negócio como Singletons, garantindo uma única instância na aplicação

Serviço	Responsabilidade
AppointmentService	Valida QR Code, busca agendamentos, valida CPF e horário
DataStorageService	Leitura/escrita de arquivos JSON no SD Card
NetworkSyncService	Autenticação na API, download de agendamentos, atualização de status
TimeService	Sincronização NTP, formatação de data/hora
FingerprintService	Gerenciamento de cadastro, verificação e sincronização de biometria digital

AppointmentService

Responsável pela lógica de validação de check-in e gerenciamento de agendamentos:

- Parse de QR Code: Extrai e valida campos do JSON (appt_id, cpf, name, hash)
- Validação de agendamento: Busca no SD Card, verifica CPF e janela de horário (±30 min)

- Verificação de status: Impede check-in duplicado verificando se já foi realizado
- Atualização de status: Marca agendamento como "realizada" localmente e na API
- Verificação biométrica: Identifica se paciente aceita biometria e se já possui digital cadastrada

DataStorageService

Responsável pelo gerenciamento de arquivos JSON no cartão SD:

- Leitura de agendamentos: Carrega arquivo JSON do dia (/appointments/YYYY-MM-DD.json)
- Escrita atômica: Atualiza arquivos JSON com novos dados de forma segura
- Busca por ID: Localiza agendamento específico dentro do array JSON
- Gerenciamento de diretórios: Cria estrutura de pastas conforme necessário

NetworkSyncService

Responsável pela comunicação com a API REST do sistema:

- Autenticação: Realiza login e obtém token JWT para requisições
- Download de agendamentos: Busca agendamentos do dia via GET /api/appointments
- Atualização de status: Envia confirmação de check-in via PUT /api/appointments/{id}
- Tratamento de erros: Gerencia timeouts, falhas de conexão e respostas HTTP
- Upload de biometria: Envia templates biométricos para POST /api/biometrics/upload
- Sincronização de check-in: Envia status de check-in com dados biométricos

TimeService

Responsável pela sincronização e formatação de horários:

- Sincronização NTP: Obtém horário preciso de servidor NTP na inicialização
- Formatação de data/hora: Converte timestamps para strings legíveis (DD/MM/YYYY HH:MM)
- Validação de janela: Verifica se horário atual está dentro do período permitido
- Fuso horário: Ajusta timestamps para o horário local brasileiro (UTC-3)

FingerprintService

Responsável pelo gerenciamento de autenticação biométrica:

- Cadastro de digital: Captura múltiplas leituras e armazena template no sensor R307S

- Verificação de digital: Compara digital capturada com templates armazenados localmente
- Sincronização de templates: Envia dados biométricos em base64 para a API
- Gerenciamento de slots: Controla posições de memória no sensor biométrico
- Validação de paciente: Identifica se paciente precisa cadastrar ou apenas verificar digital

8.5. Camada de Aplicação

Telas da Interface (UI Screens)

Tela	Arquivo	Função
Welcome Screen	welcome_screen.cpp	Tela inicial com relógio, data e instrução para escanear QR
Init Status Screen	init_status_screen.cpp	Exibe progresso de inicialização (WiFi, NTP, SD Card)
Appointment Screen	appointment_screen.cpp	Confirmação de check-in com dados do paciente e consulta
Error Screen	error_screen.cpp	Erros recuperáveis (QR inválido, consulta não encontrada)
Error Critical Screen	error_critical_screen.cpp	Erros críticos (SD ausente, WiFi indisponível)
Fingerprint Verifying Screen	fingerprint_verifying_screen.cpp	Solicita e verifica digital do paciente já cadastrado
Fingerprint Enrolling Screen	fingerprint_enrolling_screen.cpp	Cadastro de nova digital com múltiplas capturas
Fingerprint Uploading Screen	fingerprint_uploading_screen.cpp	Exibe progresso de envio dos templates biométricos para API

Máquina de Estados

O sistema possui 8 estados bem definidos:

Estado	Descrição
INITIALIZING	Conectando WiFi, sincronizando NTP, inicializando drivers
DOWNLOADING_APPOINTMENTS	Autenticando na API e baixando agendamentos
IDLE	Aguardando leitura de QR Code na Welcome Screen
VALIDATING	Processando e validando dados do QR Code

APPOINTMENT	Exibindo confirmação de check-in (timeout 5s)
ERROR	Erro recuperável, retorna para IDLE após timeout
ERROR_CRITICAL	Erro crítico, aguarda intervenção ou reinício
RESTARTING	Sistema reiniciando via watchdog
FINGERPRINT_VERIFYING	Verificando digital do paciente já cadastrado
FINGERPRINT_ENROLLING	Cadastrando nova digital com múltiplas capturas
FINGERPRINT_UPLOADING	Enviando templates biométricos para a API

Tasks FreeRTOS

O sistema utiliza 5 tasks principais executando concorrentemente:

Task	Prioridade	Stack (bytes)	Responsabilidade
task_wifi_time_init	5 (alta)	2048	Conexão WiFi e sincronização NTP no boot
task_lvgl	4	3072	Atualização periódica da interface gráfica (lv_timer_handler)
task_qr_reader	3	2048	Leitura contínua do scanner QR via UART
task_qr_validator	3	2048	Validação dos dados recebidos e atualização de estado
task_network	2 (baixa)	3072	Requisições HTTP (download/upload) em background

Mecanismos de Comunicação:

- **Queues (Filas):** qr_data_queue transfere dados do scanner (task_qr_reader) para o validador (task_qr_validator)
- **Event Groups:** wifi_time_ready_group_event sincroniza inicialização entre WiFi, NTP e LVGL

8.6. API REST Completa (endpoints, banco)

8.6.1. Endpoints da API

8.6.1.1. Autenticação

POST /auth/login

Descrição: Realiza autenticação e gera token JWT

Body:

```
{
  "username": "admin",
  "password": "senha"
}
```

Resposta:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_type": "bearer"
}
```

8.6.1.2. Pacientes

GET /pacientes

Descrição: Lista todos os pacientes com paginação

Parâmetros: skip (default: 0), limit (default: 100)

Autenticação: Requerida

GET /pacientes/{paciente_id}

Descrição: Busca paciente por ID

Autenticação: Requerida

GET /pacientes/cpf/{cpf}

Descrição: Busca paciente por CPF (11 dígitos)

Autenticação: Requerida

POST /pacientes

Descrição: Cadastra novo paciente

Body:

```
{
  "nome": "João Silva",
  "cpf": "12345678901",
  "aceita_digital": true
}
```

Autenticação: Requerida

POST /pacientes/{paciente_id}/fingerprint

Descrição: Upload de biometria digital do paciente

Content-Type: multipart/form-data

Autenticação: Requerida

GET /pacientes/{paciente_id}/fingerprint

Descrição: Download da biometria digital armazenada

Autenticação: Requerida

8.6.1.3. Categorias

GET /categorias

Descrição: Lista todas as especialidades médicas

Autenticação: Requerida

POST /categorias

Descrição: Cadastra nova categoria/especialidade

Body:

```
{
  "nome": "Cardiologia"
}
```

Autenticação: Requerida

8.6.1.4. Profissionais

GET /profissionais

Descrição: Lista todos os profissionais

Autenticação: Requerida

GET /profissionais/{profissional_id}

Descrição: Busca profissional por ID

Autenticação: Requerida

GET /profissionais/categoria/{categoria_id}

Descrição: Lista profissionais de uma especialidade específica

Autenticação: Requerida

POST /profissionais

Descrição: Cadastra novo profissional

Body:

```
{
  "nome": "Dr. Carlos Santos",
  "categoria_id": 1
}
```

Autenticação: Requerida

8.6.1.5. Horários Profissionais

GET /horarios-profissionais/{profissional_id}

Descrição: Lista horários de um profissional

Autenticação: Requerida

POST /horarios-profissionais

Descrição: Cadastra horário de atendimento

Body:

```
{
  "profissional_id": 1,
  "dia_semana": 0,
  "hora_inicio": "08:00",
  "hora_fim": "12:00",
  "duracao_minutos": 30
}
```

Autenticação: Requerida

8.6.1.6. Consultas

GET /horarios/disponiveis/{profissional_id}/{data}

Descrição: Busca horários disponíveis para agendamento

Parâmetros:

- profissional_id: ID do profissional
- data: Data no formato YYYY-MM-DD

Resposta:

```
 [
   {
```

```
    "horario": "08:00",
    "disponivel": true
  },
  {
    "horario": "08:30",
    "disponivel": false
  }
]
```

Autenticação: Requerida

POST /consultas

Descrição: Agenda nova consulta

Body:

```
{
  "paciente_id": 1,
  "profissional_id": 2,
  "data": "2026-01-15",
  "horario": "09:00"
}
```

Autenticação: Requerida

GET /consultas/paciente/{paciente_id}

Descrição: Lista consultas de um paciente

Autenticação: Requerida

GET /consultas/profissional/{profissional_id}

Descrição: Lista consultas de um profissional

Autenticação: Requerida

PATCH /consultas/{consulta_id}/status

Descrição: Atualiza status da consulta

Body:

```
{
  "status": "realizada"
}
```

Autenticação: Requerida

8.6.1.7. QR Code

GET /qrcode/generate/{consulta_id}

Descrição: Gera QR Code para check-in da consulta

Resposta:

```
{
  "consulta_id": 1,
  "qr_code": "..."
}
```

Autenticação: Não requerida

Dados codificados no QR Code:

```
{
  "appt_id": 1,
  "cpf": "12345678901",
  "name": "João Silva",
  "timestamp": "2026-01-04T10:30:00"
}
```

8.6.2. Tratamento de Erros

A API retorna códigos HTTP padrão e mensagens descritivas de erro:

- 200 OK - Operação realizada com sucesso
- 201 Created - Recurso criado com sucesso
- 400 Bad Request - Dados inválidos ou requisição malformada
- 401 Unauthorized - Token ausente ou inválido
- 404 Not Found - Recurso não encontrado
- 409 Conflict - Conflito de dados (ex: CPF já cadastrado)
- 500 Internal Server Error - Erro interno do servidor

8.6.3. Padrões Arquiteturais

O sistema adota os seguintes padrões:

- Arquitetura em Camadas (Layered Architecture)
- Repository Pattern para acesso a dados
- Dependency Injection
- RESTful API Design
- Stateful Conversation Pattern (Bot Telegram)

8.6.4. BANCO DE DADOS

8.6.4.1. Sistema de Gerenciamento

O sistema utiliza PostgreSQL como banco de dados relacional, oferecendo:

- Robustez e confiabilidade
- Suporte a transações ACID
- Armazenamento de dados binários (biometria)

8.6.4.2. Diagrama de Relacionamentos

- Paciente (1) -------- (N) Consulta
- Profissional (1) -------- (N) Consulta
- Categoria (1) -------- (N) Profissional
- Profissional (1) -------- (N) HorarioProfissional

8.6.4.3. Migrations

O sistema utiliza SQLAlchemy para gerenciamento de schema:

- Criação automática de tabelas
- Versionamento de schema
- Migrations reversíveis

8.7. BOT TELEGRAM

8.7.1. Integração com API

O bot se comunica com a API REST através de cliente HTTP dedicado, realizando as seguintes operações:

- Autenticação via JWT
- Busca de pacientes por CPF
- Cadastro de novos pacientes
- Listagem de categorias e profissionais
- Consulta de horários disponíveis
- Criação de novos agendamentos
- Geração de QR Code

8.7.2. Tratamento de Erros

- O bot implementa tratamento de erros:
- Validação de entrada do usuário
- Mensagens de erro descritivas
- Opção de reiniciar fluxo em caso de erro
- Logging de exceções
- Timeout em operações de API

8.7.3. Experiência do Usuário

O bot foi projetado para oferecer experiência intuitiva:

- Mensagens claras e objetivas
- Teclados customizados para seleção rápida
- Formatação adequada de informações
- Confirmação antes de operações críticas
- Possibilidade de cancelar operação a qualquer momento

8.8. Integração (detalhes de comunicação embarcado-API)

8.8.1. Endpoints Relevantes para Integração

GET /qrcode/generate/{consulta_id}

Geração do QR Code contendo dados da consulta

GET /pacientes/cpf/{cpf}

Validação de paciente através do CPF extraído do QR Code

GET /pacientes/{paciente_id}/fingerprint

Download da biometria armazenada para comparação

PATCH /consultas/{consulta_id}/status

Atualização do status da consulta após check-in

8.8.2. Protocolo de Comunicação

A comunicação entre embarcado e API seguirá protocolo HTTP:

- Requisições no formato JSON
- Autenticação via token JWT
- Respostas padronizadas com códigos HTTP
- Tratamento de timeout e reconexão

8.8.3. Segurança na Integração

Medidas de segurança implementadas:

- Token JWT com expiração configurável
- Validação de origem das requisições
- Hash SHA256 para validação de integridade dos QR Codes

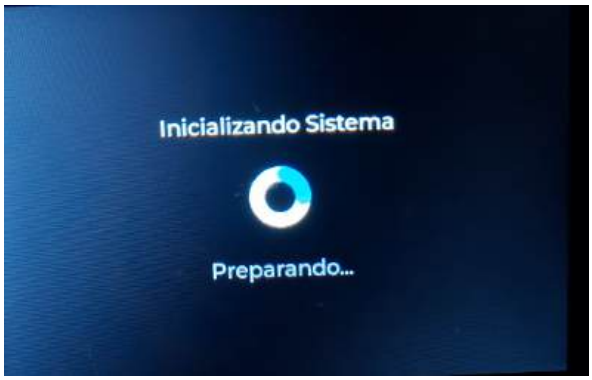
8.9. Protótipo Físico - Fotos

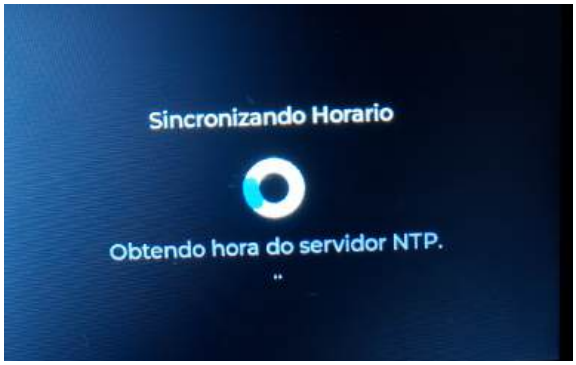
8.10. Capturas de Tela do Sistema

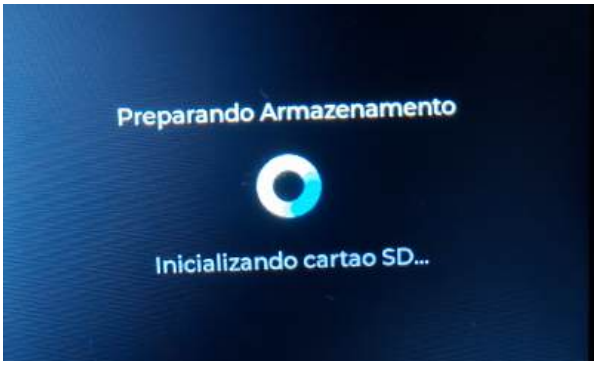
As capturas de tela a seguir ilustram as diferentes telas do sistema e os diversos cenários de uso implementados. Cada tela foi projetada para oferecer feedback claro e imediato ao usuário, seguindo princípios de design de interfaces de autoatendimento.

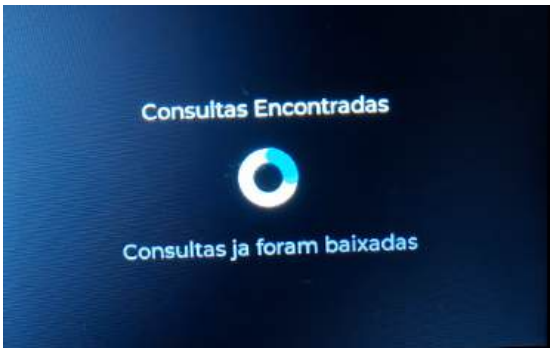
1. Tela de Inicialização (Init Status)

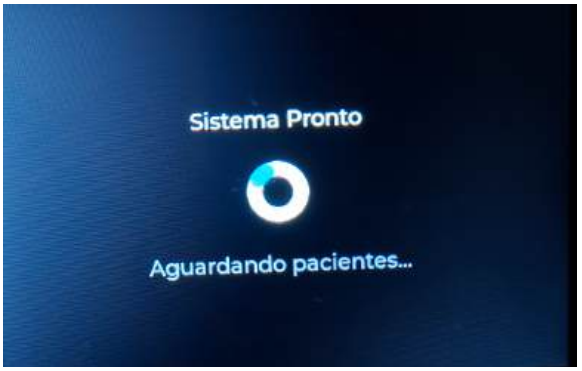
Mostra o progresso de inicialização: WiFi, NTP, SD Card, API, etc..





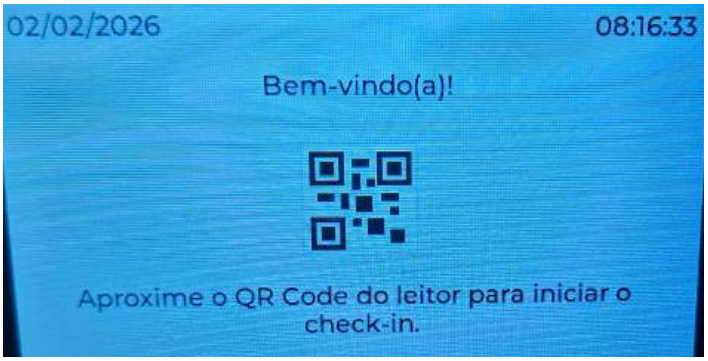






2. Tela Principal - Welcome Screen

Aguardando leitura do QR Code para iniciar o check-in.



3. Check-in Bem-Sucedido (Appointment)

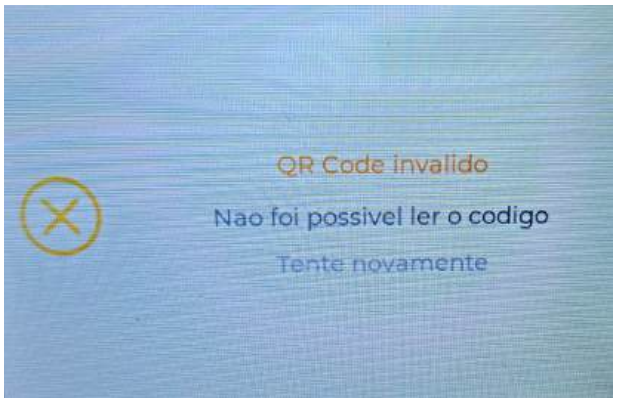
Confirmação com dados do paciente, profissional e horário agendado.

4. Autenticação Biométrica (Fingerprint)

Solicitação para cadastro ou verificação de digital.

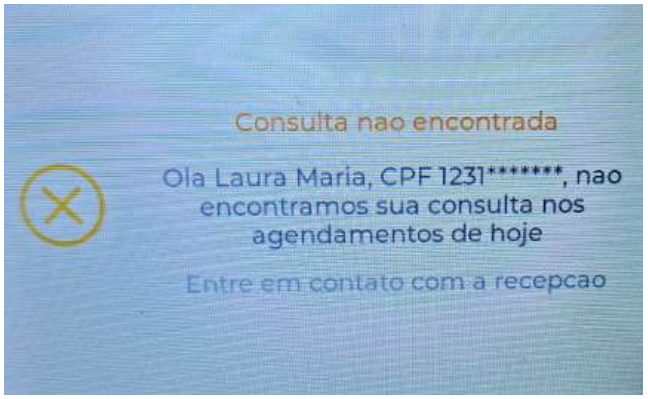
5. Erro - QR Code Inválido

Código não pode ser decodificado ou formato inválido.



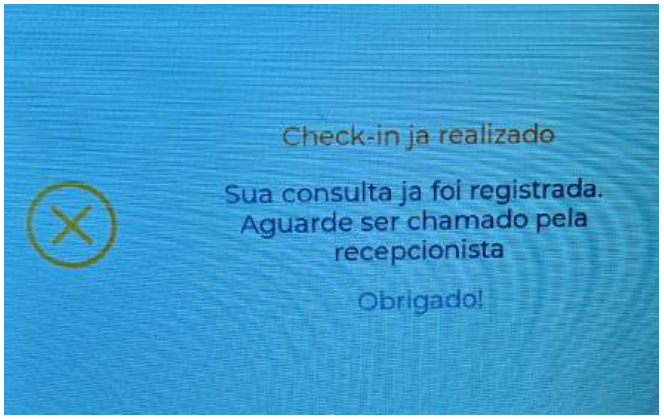
6. Erro - Consulta Não Encontrada

Agendamento não localizado na base de dados local.



7. Erro - Check-in Já Realizado

Impede duplicação: consulta já foi registrada.



8. Erro Crítico - Falha de Inicialização

Componentes essenciais indisponíveis (SD Card, WiFi, API, etc).

9. REFERÊNCIAS

1. Raspberry Pi Ltd. RP2040 Datasheet. Disponível em: https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf

2. Raspberry Pi Ltd. Raspberry Pi Pico C/C++ SDK. Disponível em: https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf

3. FreeRTOS. FreeRTOS Kernel Documentation. Disponível em: https://www.freertos.org/Documentation/RTOS_book.html

4. LVGL. LVGL Documentation v8.3. Disponível em: https://docs.lvgl.io/8.3/

5. Cesanta. Mongoose Embedded Web Server Library. Disponível em: https://mongoose.ws/documentation/

6. DaveGamble. cJSON Documentation. Disponível em: https://github.com/DaveGamble/cJSON

7. ChaN. FatFs - Generic FAT Filesystem Module. Disponível em: http://elm-chan.org/fsw/ff/00index_e.html